

A large neighbourhood search heuristic for a periodic supply vessel planning problem arising in offshore oil and gas operations

Aliaksandr Shyshou¹ Kjetil Fagerholt² Irina Gribkovskaia¹
Gilbert Laporte³

¹Molde University College, Postbox 2110, NO-6402 Molde, Norway
{Aliaksandr.Shyshou, Irina.Gribkovskaia}@himolde.no

²MARINTEK and the Norwegian University of Science and Technology,
P.O. Box 4125 Valentinlyst, NO-7450 Trondheim, Norway
kjetil.fagerholt@iot.ntnu.no

³Canada Research Chair in Distribution Management
HEC Montréal, 3000, chemin de la Côte-Sainte-Catherine
Montréal, Canada H3T 2A7
gilbert@crt.umontreal.ca

Abstract

Upstream logistics in the Norwegian oil industry includes, along with other activities, providing the offshore installations with necessary supplies. These are periodically delivered to the installations by supply vessels from onshore bases. This paper proposes a large neighbourhood search heuristic for a real-world periodic supply vessel planning problem faced by Statoil, the largest Norwegian offshore oil and gas operator. However, the model and the algorithm provided in the paper are of wide applicability. The problem calls for the simultaneous determination of a repetitive weekly sailing plan for single base and of the configuration of the supply vessel fleet. Computational comparisons with an exact solution method results demonstrate the competitiveness of the heuristic.

Keywords: maritime transportation; periodic routing; fleet sizing; multiple vehicle use; oil and gas; supply operations

1 Introduction

Oil and gas operators use supply vessels to service offshore installations from an onshore base. The assignment of installations to the supply bases is fixed by the government. Supply vessel expenses constitute a significant cost component in the area of upstream logistics as their daily charter rates may be as high as 80,000 US dollars. Supply vessel planning is crucial for ensuring continuous production and exploration as underlined in Aas et al. (2009). Our aim is to design periodic routes and schedules for a single onshore base in such a context. Our study is based on a real-world problem faced by Statoil, the largest Norwegian offshore oil and gas operator. However, the model and algorithms we provide are of wide applicability. We refer to the problem as the periodic supply vessel planning problem (PSVPP).

The PSVPP consists of simultaneously determining a repetitive weekly vessel sailing plan for a single base, made up of scheduled voyages, and the fleet configuration required to perform these voyages. Each voyage is defined by a start day and a sequence of installations to visit, both of which are not specified a priori and have to be determined. The objective is to minimize the sum of the vessel charter costs (relatively major cost component) and of the voyage sailing and service costs (relatively minor cost component). The latter costs are mostly related to the fuel consumption and are paid by the operator.

Each installation has a weekly demand that must be fulfilled. Additionally, the installations need to be visited a certain number of times per week. It is assumed that weekly demands are uniformly distributed among the visits. Some installations are closed at night, while others may be visited at any time. The starting days of vessel voyages to each installation should also be fairly evenly spread throughout the week to avoid departures on consecutive days followed by a number of days with no departures. It is more important to spread the departures to an installation than the actual visits to ensure uniform supply. It should be noted that the capacity of the onshore base is limited, i.e. the number of vessel departures on a given day is limited.

A number of practical constraints have to be respected. Voyage duration has to lie within certain limits, i.e. there are minimum and maximum durations which are measured in days. A practical restriction is that a voyage should last two or three days. This implies that a vessel can perform two to three voyages per week. There are also restrictions on the minimum and maximum number of installation visits per voyage. In addition, vessel deck capacity should not be exceeded. In reality vessels have two types of capacity: one for deck cargo and one for bulk products. However, in the vast majority of the cases, deck capacity is the constraining factor since bulk capacity is much larger compared to the demand. Additionally, all vessels depart from the base at the end of the working day when all loading operations are finished. Another practical restriction is that there should be no vessel departures on certain weekend days.

In Figure 1 we depict a weekly sailing plan for the Mongstad base involving three vessels (Star, Symphony, Foresight) and five offshore installations (BID, BRA, DSD, HDA, GRA). The number in each cell of the second row corresponds to the end of an eight-hour slot. In this example each vessel performs two voyages during the week. The voyages have a bold outline in the figure. Each voyage is preceded by an eight-hour period at the onshore base, when loading and unloading operations take place. In the figure these peri-

ods are shaded with diagonal stripes. It can be seen that vessel Foresight starts its second voyage on Saturday at 16:00 (144 hours after the beginning of the week) and finishes it on Monday of the following week. A comment regarding the calculation of voyage duration measured in days is appropriate at this point. If d is the duration of a voyage in hours, then the duration of the same voyage in days is defined as $\lceil d/24 \rceil$. This means that a vessel performing a voyage starting e.g. on Monday and lasting 41 hours (plus eight-hour turn-around time at the base) lasts three days and can start a new voyage only on Thursday, while if the voyage lasts less than 40 hours, a new voyage can be started on Wednesday thus avoiding long idle period.

	Monday			Tuesday			Wednesday			Thursday			Friday			Saturday			Sunday		
	8	16	24	32	40	48	56	64	72	80	88	96	104	112	120	128	136	144	152	160	168
Star			GRA		BID	DSD		BRA			GRA			BID		BRA					
Symphony					GRA	HDA			BID	DSD	BRA				BID	DSD	BRA				
Foresight	DSD	BRA						GRA			BID	DSD					GRA	HDA		BID	

Figure 1: Example of a weekly sailing plan

Geographical positions of the Mongstad base and of the five installations can be found in Figure 2. We also illustrate the voyage starting on Monday.

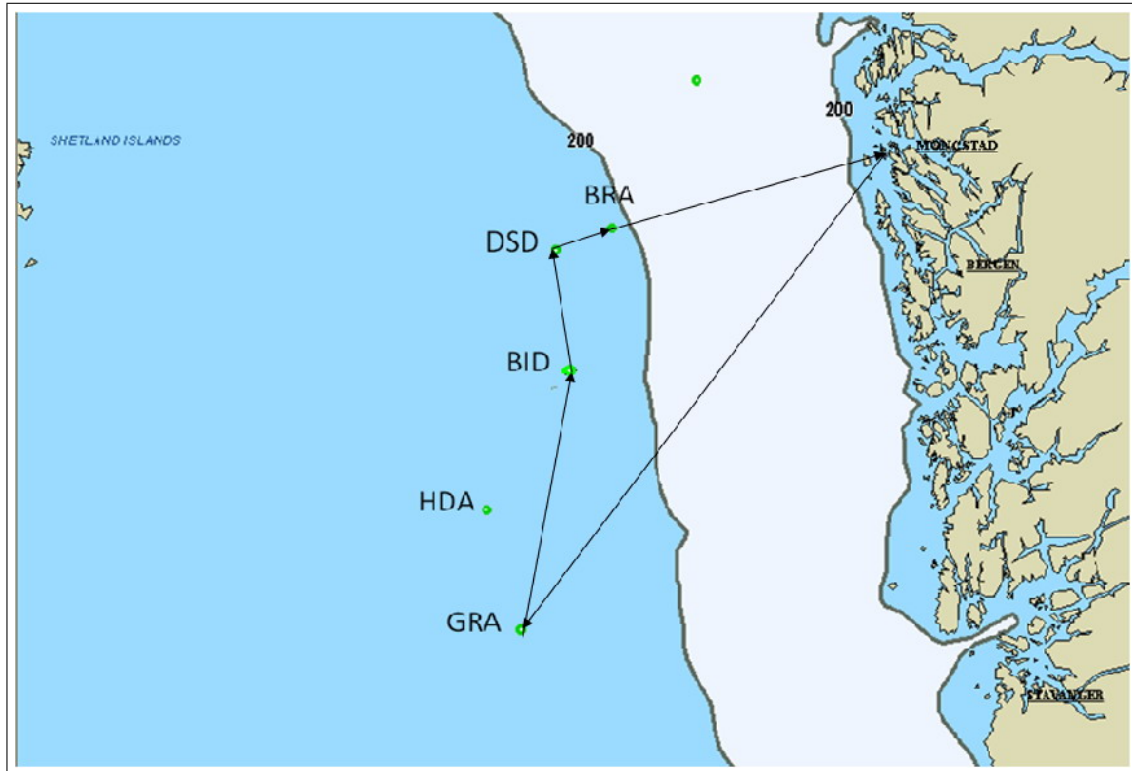


Figure 2: Positions of the installations and the supply base on the map

The PSVPP can be classified as a periodic vessel routing and fleet sizing problem with time windows and multiple use of vessels. A number of problem aspects stand out from the classification, namely periodic routing, fleet sizing and mix, and multiple use of vehicles during the multi-period planning horizon. The PSVPP may be regarded as a two-level bin packing problem, where we want to combine first-level items (installation visits) into second-level items (voyages) of a certain weight each (service and sailing cost). These second level items have to be packed in a number of bins (vessels) such that the resulting packing is the best with respect to the sum of bin weights (vessel charter costs) and the total weight of second-level items (total service and sailing costs). Thus, the problem we study involves routing, scheduling and packing decisions.

The literature on periodic vehicle routing problem is quite extensive. Notable references include Cordeau et al. (1997), Cordeau et al. (2001), Cordeau et al. (2004), Delgado et al. (2005), Polacek et al. (2007), Alegre et al. (2007), and Hemmelmayr et al. (2009). An important distinction of the problem we study is that the duration of the voyage may be up to three days, whereas a provision commonly shared by the periodic routing literature is that no route can last more than one day. These types of problems only involve routing and scheduling decisions.

The vehicle routing problem with multiple use of vehicles (VRPMUV) was studied by a number of authors. These types of problems involve routing and packing decisions. Fleischmann (1990), Taillard et al. (1996), Azi et al. (2006) and Olivera and Viera (2007) all use a two-stage solution method. In the first stage a set of either “good” or non-dominated VRP solutions is produced using either heuristic or exact algorithms. In the second stage these routes are used as “building blocks” to produce solutions to the VRP-MUV in which vehicles are possibly used more than once during the planning horizon. A different approach is adopted by Brandão and Mercer (1997) and Brandão and Mercer (1998). The authors construct an initial solution by a combination of nearest neighbour and cheapest insertion heuristics, which is then improved by a tabu search algorithm.

Relaxed versions of the PSVPP have been studied by Fagerholt and Lindstad (2000) and Gribkovskaia et al. (2007). While Gribkovskaia et al. (2007) mainly focused on the routing aspect of the problem, Fagerholt and Lindstad (2000) simultaneously determined which vessels to operate and the respective weekly schedules. In Fagerholt (1999) a similar approach is used to determine optimal fleet and weekly routes for selected ships in a liner ship routing problem.

The previous studies did not simultaneously consider all the complicating aspects of the PSVPP. The only exception is the recent study of Halvorsen-Weare et al. (2009), who have proposed two exact methods for the PSVPP. The first uses an arc flow model, while the second is an extension of the method used by Fagerholt and Lindstad (2000) and is based on the pregeneration of all cheapest feasible voyages which are then used as an input to a set covering model with numerous side constraints. The authors demonstrate that the second method called the voyage-based formulation, is computationally superior to the first one when solved using a commercial solver. For large instances a heuristic is derived by halting the search after a preset time and reporting the best known feasible solution.

In this paper we propose an original large neighbourhood search heuristic capable of solving a difficult planning problem involving routing, scheduling and bin packing

decisions. We evaluate its performance by comparing heuristic solutions to those of Halvorsen-Weare et al. (2009). The remainder of the paper is organized as follows. In the next section a generalization of the voyage-based formulation of Halvorsen-Weare et al. (2009) is presented to avoid any ambiguity regarding the definition of the PSVPP. This is followed by the description of the large neighbourhood search heuristic and by its computational evaluation. Finally, some conclusions are drawn.

2 Mathematical model

In this section we present a generalization of the voyage-based formulation of Halvorsen-Weare et al. (2009) for the PSVPP. Let N be the set containing all offshore installations and let V be the set of available supply vessels. Further, let T be a circularly ordered set representing the days of the planning horizon (a week in our application), and let R_j be the set of pregenerated cheapest feasible voyages vessel $j \in V$ may sail. Let L be a set of all possible voyage durations measured in days and let F be a set of all possible visit frequency values. Additionally, let R_{jl} be a set of pregenerated cheapest feasible voyages for vessel $j \in V$ with a duration of l days ($l \in L$) and let N_r be a subset of the installations requiring r visits per week ($r \in F$). The formulation then uses the following notation: c_j is the weekly charter cost for using vessel $j \in V$; b_t is an upper bound on the number of vessels leaving the onshore base on day $t \in T$; c_{jk} is the precalculated sailing and service cost associated with vessel $j \in V$ sailing voyage $k \in R_j$; s_i is the required number of visits per week for installation $i \in N$; a_{ijk} are binary constants ($i \in N, j \in V, k \in R_j$) equal to 1 if and only if vessel j visits installation i on voyage k . The parameter d_{jk} , rounded up to the nearest integer, represents the duration (in days) of voyage k sailed by vessel j ($j \in V, k \in R_j$); f_j is the number of days vessel j is available for during the planning horizon ($j \in V$). We define $0 \leq h_r \leq |T|$ to be the length of an auxiliary sub-horizon for the installations whose visit frequency is r . We further define $\bar{p}_r$ and $\underline{p}_r$ as an upper and a lower bound on the number of visits during the sub-horizon h_r an installation whose visit frequency is r ($r \in F$) should receive.

The decision variables of the model are:

$$x_{jkt} = \begin{cases} 1, & \text{if vessel } j \text{ starts voyage } k \text{ on day } t \text{ } (j \in V, k \in R_j, t \in T) \\ 0, & \text{otherwise} \end{cases}$$

$$y_j = \begin{cases} 1, & \text{if vessel } j \text{ is used } (j \in V) \\ 0, & \text{otherwise.} \end{cases}$$

The mathematical model is then:

$$\text{minimize } \sum_{j \in V} c_j y_j + \sum_{j \in V} \sum_{k \in R_j} \sum_{t \in T} c_{jk} x_{jkt} \quad (1)$$

subject to

$$\sum_{j \in V} \sum_{k \in R_j} \sum_{t \in T} a_{ijk} x_{jkt} \geq s_i, i \in N \quad (2)$$

$$\sum_{k \in R_j} \sum_{t \in T} d_{jk} x_{jkt} - f_j y_j \leq 0, j \in V \quad (3)$$

$$\sum_{j \in V} \sum_{k \in R_j} x_{jkt} \leq b_t, t \in T \quad (4)$$

$$\sum_{k \in R_{jl}} x_{jkt} + \sum_{k \in R_j} \sum_{q=1}^{l-1} x_{jk, (t+q) \bmod |T|} \leq y_j, j \in V, t \in T, l \in L \quad (5)$$

$$\underline{p}_r \leq \sum_{j \in V} \sum_{k \in R_j} \sum_{h=0}^{h_r} a_{ijk} x_{jk, (t+h) \bmod |T|} \leq \bar{p}_r, i \in N_r, t \in T, r \in F \quad (6)$$

$$y_j \in \{0, 1\} \quad (7)$$

$$x_{jkt} \in \{0, 1\}. \quad (8)$$

The objective function (1) minimizes the sum of vessel charter costs and of voyage sailing and service costs. Constraints (2) ensure that each installation receives the required number of visits per week. Constraints (3) limit the usage of a vessel to the number of days for which it is available, while constraints (4) guarantee that an upper bound on the number of vessels leaving the onshore base on a given day is not exceeded. Constraints (5) mean that a vessel (which is used) starting a voyage of l -day duration on day t cannot start a voyage on days from $t + 1$ to $t + l - 1$. Constraints (6) ensure the uniformity of departures to the installations, depending on their visit frequency. For example let us consider $r = 3$: corresponding parameter values are $h_3 = 3$, $\bar{p}_3 = 2$ and $\underline{p}_3 = 1$, which results in a constraint that requires at least one departure and no more than two departures every three days for an installation whose visit frequency is three. Constraints (7) and (8) impose the binary requirements on the variables.

3 Large neighbourhood search heuristic for the PSVPP

In this section we present a large neighbourhood search heuristic (LNS) for the PSVPP. This type of algorithm, where at each iteration a large portion of solution is rearranged, was first proposed by Shaw (1998). An overview of the algorithm for the PSVPP is first given, followed by the description of the initial constructive heuristic. Simple improvement procedures, an integral part of the LNS heuristic, are then provided. Finally, the detailed description of the heuristic itself is presented.

3.1 Overview of the heuristic

The heuristic is applied for a given number of restarts. At each restart we randomly generate initial feasible solution and then perform a number of LNS iterations. At each iteration, the neighbourhood $N(z)$ of a solution z is defined as the set of all solutions reachable from z by taking a fixed number of voyages, removing from each a randomly generated number of visits, and reinserting them into voyages using regret-like heuristics. A transition from z to neighbour solution z' is called a *move*.

An LNS iteration starts with a random removal of several visits from a number of voyages and putting them in a bank of uninserted visits S . The visits from S are then reinserted back into voyages (not necessarily into voyages from which they were removed) using regret insertion heuristics. If $S \neq \emptyset$ the algorithm performs the next move.

If $S = \emptyset$ after the reinsertion stage we apply several local improvement procedures. We first apply those attempting to reduce the number of vessels used (since this is a relatively major cost component). Initially we attempt to reduce the number of voyages and then try to reduce the number of vessels by a better packing of remaining voyages to vessels. We proceed to reducing the total duration of voyages measured in days and again try to pack voyages in a smaller number of vessels. Finally, a procedure minimizing total sailing and service costs is applied, which performs visit relocations between voyages. Feasibility is maintained when applying local improvement procedures, i.e. S is kept empty.

After the local improvement stage an attempt is made to further reduce the number of vessels used. If a preliminary check indicates that this reduction may be possible, we identify and implement the reassignment of voyages to vessels resulting in the least number of days one of the vessels (say j^*) is used for. We then remove all visits from the voyages vessel j^* performs, putting them in S , and set the status of the vessel j^* to "not used". If the resulting number of voyages is not larger than the lower bound on the number of voyages in the sailing plan, we try to create one or several empty voyages for visit insertions at later iterations and assign these voyages to vessels that are already used. The lower bound on the number of voyages in the sailing plan is defined as

$$\underline{n} = \max \left\{ s_{\max}, \left\lceil \frac{D}{Q_{\max}} \right\rceil, \left\lceil \frac{\lambda_{\text{total}}}{\lambda_{\max}} \right\rceil \right\}, \quad (9)$$

where $s_{\max}$ is the maximum value of an installation visit frequency for a given problem instance, D is the total weekly demand of all installations, $Q_{\max}$ is the capacity of the largest vessel, λ_{total} is the total number of visits for an instance, and $\lambda_{\max}$ is the maximum allowed number of visits per voyage. Expression (9) states that the number of voyages in a feasible solution of a given problem instance cannot be smaller than the maximum of three values: 1) the largest visit frequency value as each installation only receives up to one visit per voyage; 2) the total demand of all installations divided by the capacity of the largest vessel rounded up to the nearest integer; 3) the total number of visits divided by the maximum number of visits per voyage rounded up to the nearest integer.

It should be noted that empty voyages are only created if after removing the voyages of vessel j^* and putting removed visits into S the total number of voyages is no larger than $\underline{n}$. Otherwise, empty voyages are not created to avoid sparse sailing plans, i.e. sailing plans with larger number of voyages, but with fewer visits defining each voyage.

3.2 Initial constructive heuristic

A solution to the PSVPP consists of a collection of voyages and of an assignment of available vessels to these voyages. To construct an initial feasible solution the following procedure is executed:

- A feasible pattern of departure days is randomly generated for each offshore installation. The pattern should be feasible with respect to the installation visit frequency

and the uniformity of departures.

- Then, for each departure day $t \in T$ voyages are constructed:
 - Let λ_t be the number of visits to be made by vessels departing on day t and let $\lambda_{\max}$ be the maximum allowed number of visits for a voyage.
 - **If** $\lambda_t \leq \lambda_{\max}$, a single voyage for all these visits is constructed.
 - **Else** the visits are evenly distributed between $\lceil \lambda_t / \lambda_{\max} \rceil$ voyages. The number of voyages should not exceed the maximum number of departures per day.
 - **End if**
 - The duration of each voyage is then reduced by means of an intra-voyage optimization routine to be described shortly. Voyage durations should lie within specified limits.
- After the voyages have been constructed, an attempt is made to assign vessels to these voyages. For each voyage, starting from Monday, the smallest available vessel with sufficient capacity is assigned.

If at some point there is no available vessel with sufficient capacity, the entire procedure is repeated until a feasible solution is found. A feasible solution consists of a collection of feasible voyages and of a feasible assignment of voyages to vessels. The assignment is feasible if each used vessel performs a number of non-overlapping voyages and on each voyage the vessel capacity is sufficient to accommodate the total demand for all visited installations. In a feasible solution the number of vessel departures each day should not exceed the capacity of the base.

3.3 Improvement procedures

This section is devoted to improvement procedures for the PSVPP. The solution is kept feasible at all times.

3.3.1 Intra-voyage optimization

To reduce the duration of a voyage we remove one visit at a time (starting from the beginning of the voyage) and reinsert it into the first position which reduces the voyage duration (first-accept strategy). This removal-reinsertion operation is repeated until no further improvement is found.

3.3.2 Reducing the number of voyages

This routine performs the following steps for each voyage:

- Remove one visit at a time from the voyage and try to insert it into another voyage.
- Accept the changes only if this can be done for *all* visits of the voyage without solution feasibility violation.

The procedure stops if the number of voyages cannot be further reduced or if the lower bound on the number of voyages ($\underline{n}$) is reached.

3.3.3 Reassigning vessels to voyages

The purpose of this procedure is twofold: reducing the fleet size by a better packing of voyages to vessels and reducing the average capacity of the fleet by keeping its size constant. The procedure repeats the following steps for every vessel used in the weekly sailing plan:

- Try to reassign each voyage performed by the vessel to another used vessel or to a smaller (not necessarily used) vessel maintaining solution feasibility.
- Accept the changes only if *all* voyages performed by the vessel can be reassigned.

3.3.4 Relocating visits to other voyages

The steps of this routine are as follows:

- **For** each origin voyage k (a voyage from which visits are relocated is referred to as origin voyage)
 - **For** each visit v belonging to k
 - Evaluate all feasible relocations of v to other voyages. The relocation is feasible if:
 - It does not violate vessel capacity of the destination voyage, i.e. the voyage to which the visit is relocated.
 - A uniform spread of departures is maintained for the installation whose visit is being relocated.
 - The durations of origin and destination voyages after the relocation are within their lower and upper bounds.
 - **End for**
- **End for**
- Implement the relocation yielding the largest decrease in the objective (if any).
- **Repeat** this procedure as long as there exist relocations improving the objective function.

3.3.5 Reducing total duration of the voyages measured in days

Applying this procedure helps to reduce some voyage durations measured in days, even if the reduction is only a few hours (recall that a 39-hour voyage lasts two days, while a 41-hour voyage lasts three days). The effect of this operation is an increased vessel availability. The procedure performs the following steps:

While there are voyages whose duration can be reduced:

- **For** every voyage whose duration in days is larger than the minimum duration:
 - **While** there are feasible visit relocations or until the duration of the voyage is reduced by one day:
 - For each visit belonging to the voyage evaluate all feasible relocations to different voyages so that the duration of the destination voyage in terms of number of days does not increase.
 - Implement the best (the smaller the better) relocation in terms of the following lexicographic ordering: 1) number of feasible relocations (to prioritize the relocation of visits which are "difficult" to relocate); 2) net voyage duration increase (destination voyage duration increase minus origin voyage duration decrease). 3) increase in the objective function.
 - **End while**
 - If voyage duration is reduced, store the respective change in the objective.
- **End for**
- Implement the voyage duration reduction yielding the smallest objective function increase.

End while

3.4 Large neighbourhood search heuristic

We now proceed to the description of the large neighbourhood search heuristic for the PSVPP. The outline of our algorithm is the following:

For ρ restarts do:

- Randomly generate initial feasible solution with the initial constructive heuristic.
- Set $\vartheta = 0$.
- **For** η iterations do:
 - *Beginning of removal stage*
 - Randomly choose a subset θ of voyages, from which several visits will be removed.
 - **For** each voyage $k \in \theta$ repeat:
 - Denote by w_k the number of visits belonging to voyage k .
 - Randomly generate the number of visits $\mu \in [1, w_k - 1]$ to be removed from voyage k .
 - Randomly select $l \in [1, w_k]$.

-
- **If** $\mu < w_k - l$, remove μ consecutive visits starting with the installation in the l^{th} position of voyage k .
 - **Else** remove $w_k - l$ consecutive visits starting with the installation in the l^{th} position of voyage k and remove $\mu - w_k + l$ consecutive visits starting from the beginning of the voyage.
 - **End if**
 - Insert all removed visits in a pool S .
 - Apply the intra-voyage optimization procedure to the remaining visits of voyage k .
 - **End for**
 - *End of removal stage*
 - **If** $\vartheta = 1$ (i.e. if we removed some voyages when trying to reduce the number of vessels used at one of the previous iterations and the resulting number of voyages is no larger than $\underline{n}$):
 - *Beginning of empty voyage creation stage*
 - For every vessel used try to find a time slot where a new voyage may be created. The slot has to be:
 - At least voyage minimum duration (in days) long.
 - The vessel has to be available during this time slot.
 - Previous voyage performed by the vessel has to be at least minimum voyage duration (in days) long.
 - Create an empty voyage in every time slot found for possible future insertions.
 - If at least one empty voyage was created set $\vartheta = 0$.
 - *End of empty voyage creation stage*
 - **End if**
 - *Beginning of visit reinsertion stage*
 - **While** there are feasible insertions of visits from S or until $S = \emptyset$:
 - **For** every visit v in S :
 - **For** every voyage k determine whether v can be feasibly inserted into voyage k . If so, tentatively insert v at the beginning of voyage k , apply the intra-voyage optimization procedure, and store the corresponding insertion cost and the destination voyage duration increase. If not, proceed to the next voyage.
 - **End for**
 - If there are several feasible insertion positions, calculate the regret values for each insertion, i.e. the differences between the second best insertion cost and the best insertion cost.
 - **End for**
 - Identify the best insertion (say, visit v^* into voyage k^*):

-
- **If** there are visits in S with only one feasible insertion voyage, then define v^* as the visit yielding the smallest values of the lexicographic ordering: 1) duration increase (in days) of the destination voyages; 2) objective function increase. Let k^* be the voyage in which v^* is inserted.
 - **Else if** all visits in S have more than one feasible insertion position, then define v^* as the visit with the smallest duration increase (in days) and largest regret value ordered lexicographically. Let k^* be the voyage in which v^* is inserted.
 - **End if**
 - Insert v^* at the beginning of k^* and apply to k^* the intra-voyage optimization procedure.
 - Set $S = S \setminus \{v^*\}$.
 - **End while**
 - *End of visit reinsertion stage*
 - **If** $S = \emptyset$ and the solution is feasible:
 - *Beginning of local improvement stage*
 - **While** there is objective function decrease apply improvement procedures (first those attempting to reduce vessel charter costs, then those reducing the sailing costs):
 - Reduce the number of voyages.
 - Reassign vessels to voyages.
 - Reduce total duration of the voyages measured in days.
 - Reassign vessels to voyages.
 - Relocate visits to other voyages.
 - **End while**
 - *End of local improvement stage*
 - If the best solution found so far is improved, update it.
 - For current sailing plan let s be the sum of all voyage durations (in days), m be the total number of vessels used, and f_{min} be the minimum number of days one of the used vessels is available (typically all vessels are available for $|T|$ days, so f_{min} will also be $|T|$).
 - **If** $\lceil s/f_{min} \rceil < m$, try to reduce the number of vessels used:
 - *Beginning of fleet size reduction stage*
 - For every vessel used evaluate partial reassignments of the voyages it performs (i.e. reassignments of some of the voyages, but not necessarily of all voyages) to other vessels.
 - Implement the partial reassignment resulting in the least number of days one of the vessels (say j^*) is used for.
 - Remove all visits from the voyages vessel j^* performs and put them in S .

-
- Set the status of vessel j^* to "not used".
 - If the resulting number of voyages is no larger than $\underline{n}$, set $\vartheta = 1$.
 - *End of fleet size reduction stage*
 - **End If**
 - **End if**
 - **End for**
 - End for**

As can be seen from this description, there may be several uninserted visits at the end of an iteration. However, as the initial solution is feasible, the algorithm always returns a feasible solution.

4 Computational experiments

In this section we first describe the test instances. We then proceed to the computational evaluation of the large neighbourhood search heuristic.

4.1 Test instances description

Halvorsen-Weare et al. (2009) have evaluated their solution methods on 31 single-base instances, based on actual data provided by Statoil. These instances vary in size with the smallest involving three installations and the largest involving 14 installations. The name of an instance typically consists of four numbers separated by a hyphen: the first number is the number of installations; the second number is the number of available vessels; the third number is the total number of installation visits; the fourth number is the number of installations closed between 19:00 and 7:00.

Halvorsen-Weare et al. (2009) also report computational experience on instances involving several real-life complicating aspects of the PSVPP, which Statoil has to deal with. Frequently, some installations may require visits by vessels departing from the supply base on specific days. In particular, installations 4 and 5 require a visit by a vessel departing on Tuesday, installation 6 by vessels departing on Tuesday and Friday, and installation 10 by a vessel departing on Wednesday. Taking instance "11-5-47-3" as a base case and following the notation of Halvorsen-Weare et al. (2009), we refer to this instance as "11-5-47-3-A".

Another complicating real-life aspect of the PSVPP is related to voyage duration. So far, we have considered that duration of a voyage was at least two and at most three days. In practice, however, voyages lasting only two days are preferred, and three-day voyages may only start on Friday or Saturday. Introducing these considerations defines instance "11-5-47-3-B". Instance "11-5-47-3-AB" combines the characteristics of "11-5-47-3-A" and "11-5-47-3-B".

The formulation we have presented does not prevent several vessels from being at the same installation at the same time. We refer to such a situation as a collision. More

specifically, we say that two neighbouring visits are colliding if the first visit ends less than p hours before the beginning of the second visit. Following the lines of Halvorsen-Weare et al. (2009) we have used $p = 6$. Forbidding collisions defines the instance "11-5-47-3-col", where the complicating aspects of instances "11-5-47-3-A" and "11-5-47-3-B" are kept.

Unexpected events such as rough weather conditions may cause delays to planned voyages. For this reason, very tight schedules, where delays on one route may affect other routes, are undesirable in practice. As a result, schedules with a certain slack are preferred, where the slack is defined as the time between the end of a voyage performed by a given vessel and the start of the next voyage of this vessel. Clearly, the number of slack periods is equal to the number of voyages in the weekly schedule, and the minimum slack, as the name suggests, is the minimum of the slack values. Requiring a minimum slack of at least h hours ensures a certain degree of robustness in the weekly sailing plan. Instances "11-5-47-3-S2" and "11-5-47-3-S4" are obtained by setting h to two and four hours respectively, and keeping all previously defined complicating aspects.

Instances 11-5-47-3-I to 11-5-47-3-III are obtained by taking instance "11-5-47-3-col" as the base case and analyzing the following scenarios:

- 11-5-47-3-I: under this scenario all installations are open for service 24 hours.
- 11-5-47-3-II: under this scenario installations 1, 8 and 9 receive five visits a week instead of six.
- 11-5-47-3-III: under this scenario installation 4 is visited from a different supply base.

Clearly, the difficulty of the PSVPP is mostly related to the problem size, namely to the number of installation visits.

4.2 Assessment of the large neighbourhood search heuristic

In this section we present the computational results for our LNS heuristic and compare them to those obtained with the voyage-based formulation (VBF) of Halvorsen-Weare et al. (2009) implemented using a commercial solver.

There are three user-defined parameters in the LNS heuristic: 1) the number of restarts (ρ); 2) the number of LNS iterations at each restart (η); 3) the number of voyages from which several visits are removed (the cardinality of θ). For computational evaluation we distinguish between smaller instances (up to eight installations), medium instances (nine and ten installations), and larger instances (more than ten installations). After some preliminary experimentation the (ρ, η) couples were set to (10,10) for smaller instances, (10,75) for medium instances, and (10,200) for larger instances. Choosing θ of cardinality two proved to be a good trade-off between solution quality and computation time.

Computational results for basic instances are reported in Table 1. VBF was implemented in Xpress-IVE 1.19.00 with Xpress-Optimizer 19.00.00 as a solver, but not all instances could be solved to optimality within the allocated CPU time of 3,600 seconds. This time budget is only for the solution of the integer linear program; the pregeneration of all voyages takes additional time. Column "**Gap (%)**" gives the optimality gap

for the two methods. When the problem instance is solved to optimality, this gap is zero for VBF. Otherwise the percentage gap between the value of the best found solution and that of the smallest valid lower bound is reported. The LNS gap is computed using the optimal solution when VBF gap is zero and using the best known lower bound (+) in the truncated search tree produced by VBF when it is not. Column "Seconds" gives the CPU time in seconds for both methods; this value for VBF includes voyage pregeneration time. Column "Voyages" reports the number of voyages, while the chosen vessel fleet size and mix is the same for both methods in all problem instances.

We observe that LNS produces optimal or quasi-optimal solutions for the instances with up to twelve installations. On small and medium size instances, VBF is slightly better and much faster than LNS. However, as instance size grows, VBF, which requires the enumeration of all cheapest feasible voyages, gradually becomes less effective because of its fast growing computation time. In comparison, the computation time of LNS is more stable and this heuristic starts dominating VBF on the more difficult instances involving 14 installations. No comparative data are available for larger instances but our results indicate that LNS can still be applied when VBF ceases to be practical.

Instance	Gap (%)			Seconds		Voyages	
	VBF	LNS		VBF	LNS	VBF	LNS
3-5-16-0	0.00	0.00		0.0	5.2	6	6
4-5-21-0	0.00	0.00		0.1	13.9	6	6
5-5-23-0	0.00	0.00		1.4	21.9	6	6
5-5-23-1	0.00	0.00		0.4	23.1	6	6
6-5-25-0	0.00	0.00		1.1	30.0	6	6
6-5-25-2	0.00	0.03		0.8	32.4	6	6
7-5-30-0	0.00	0.00		3.0	55.6	6	6
7-5-30-2	0.00	0.06		1.9	65.3	6	6
8-5-36-0	0.00	0.00		5.6	109.2	6	6
8-5-36-2	0.00	0.02		18.4	114.3	6	6
9-5-42-0	0.00	0.01		51.1	695.2	6	6
9-5-42-2	0.00	0.05		53.2	663.9	6	6
10-5-43-0	0.00	0.03		251.5	657.5	7	7
10-5-43-3	0.00	0.01		275.2	671.0	7	7
11-5-47-0	0.00	0.18		2 670.0	1 696.4	8	8
11-5-47-3	0.00	0.17		2 078.2	1 927.4	8	8
12-5-51-0	0.00	0.13		1 889.1	1 994.7	8	8
12-5-51-3	0.00	0.19		872.7	2 253.5	8	8
13-5-55-0	17.33	18.19	(+)	4 483.4	2 248.4	8	9
13-5-55-3	14.75	15.67	(+)	4 492.9	2 384.5	8	9
14-5-59-0	14.86	14.14	(+)	7 499.6	3 376.4	10	9
14-5-59-3	19.34	16.22	(+)	7 671.9	2 857.7	9	9
Average	3.01	2.96		1 469.1	995.3	6.95	7.00

Table 1: Computational results for basic instances

We now analyze our results for the instances of the PSVPP with practical complicating aspects. It should first be noted that incorporation of these aspects into VBF required additional constraints. Several minor modifications have also been made to our LNS algorithm to solve these instances. Every occurrence of $\lambda_{\max}$ is replaced by $(\lambda_{\max} - 1)$ in the initial constructive heuristic. This is done to have more voyages in the initial solution since the constraints on voyage duration are stricter. Moreover, the initial constructive heuristic no longer guarantees the feasibility of the produced solution with respect to the duration of the voyages starting on Monday to Thursday. For this reason the selection of a subset of θ voyages from which to remove several visits is no longer randomized. If there are voyages violating the duration limit, several visits are removed from these. Only if there are none is the selection randomized.

To counteract possible collisions, an additional module was added to the LNS heuristic. The module performs an iterative post-processing of an otherwise feasible solution in the following way:

- **While** there are colliding visits or until the number of iterations does not exceed γ :
 - **For** each pair of colliding visits evaluate the following options:
 - reverse the orientation of the voyage the first colliding visit belongs to;
 - reverse the orientation of the voyage the second colliding visit belongs to;
 - reverse the orientation of both voyages.
 - **End for**
 - Implement the best modification in terms of the following lexicographic ordering: 1) number of colliding visit couples; 2) change in the solution cost.
- **End while**

Setting $\gamma = 10$ proved sufficient in our tests. Computational results for instances with various practical aspects are summarized in Table 2. Keeping the total number of LNS iterations of 2000, the value for the (ρ, η) couple has been set to (50,40). This is done to diversify the search and to avoid spending much time searching for a three-vessel sailing plan as most of the instances of Table 2 due to practical complicating aspects do not have a 3-vessel feasible solution. We also use the optimal objective value for instance "11-5-47-3-AB" as a lower bound for instances "11-5-47-3-col", "11-5-47-3-S2" and "11-5-47-3-S4", as VBF is itself a heuristic whenever collisions are involved. To guarantee optimality with collision constraints one has to pregenerate all possible voyages, while in VBF only a subset of cheapest voyages of both orientations is pregenerated.

By and large, the comments made regarding the results of Table 1 still apply to those of Table 2. The relatively larger gaps for instances "11-5-47-3-AB", "11-5-47-3-col", "11-5-47-3-I", and "11-5-47-3-II", can be explained by the inability of the LNS to sufficiently reduce the number of voyages as it was primarily designed for the basic version of the problem. However, LNS still outperforms VBF on instance "11-5-47-3-S4".

Instance	Gap (%)			Seconds		Voyages	
	VBF	LNS		VBF	LNS	VBF	LNS
11-5-47-3-A	13.18	14.39	(+)	3 673.4	2 557.7	7	8
11-5-47-3-B	0	0.36		145.9	2 546.0	9	9
11-5-47-3-AB	0	0.62		80.5	2 535.2	9	10
11-5-47-3-col	0.01	0.76	(+)	255.5	2 398.1	9	10
11-5-47-3-S2	0.65	0.76	(+)	3 673.4	2 304.5	10	10
11-5-47-3-S4	1.76	1.06	(+)	3 673.4	2 558.4	11	10
11-5-47-3-I	0	0.85		112.9	1 747.0	9	10
11-5-47-3-II	0	0.98		153.8	1 616.2	9	10
11-5-47-3-III	0	0.43		114.9	1 761.8	8	8
Average	1.73	2.25		1 320	2 225	9.00	9.44

Table 2: Computational results for additional instances involving real-life complicating aspects of the problem

5 Conclusion

We have presented a large neighbourhood search heuristic for the periodic supply vessel planning problem. The problem, consisting of finding scheduled voyages for a single base and the mix of vessels to perform these voyages, was inspired by the operations of Statoil, but it is also encountered in several other distribution management contexts. Supply vessels are a costly resource and the problem is a considerable source of challenges in the area of upstream logistics for oil and gas operators. Computational experiments, based on real data from Statoil, demonstrate that the heuristic is quite efficient for the instances considered and can be used to solve larger instances of the problem within reasonable time.

Acknowledgements. We are grateful to Elin Espeland Halvorsen-Weare for providing her instances and solutions. This work was partially supported by the Canadian Natural Sciences and Engineering Research Council under grant 39682-05 and by the Research Council of Norway through the DESIMAL project. This support is gratefully acknowledged.

References

- Aas B, Halskau Ø sr, Wallace S W (2009). The role of supply vessels in offshore logistics. *Maritime Economics & Logistics* **11**: 302–325.
- Alegre J, Laguna M, Pacheco J (2007). Optimizing the periodic pick-up of raw materials for a manufacturer of auto parts. *European Journal of Operational Research* **179**: 736–746.
- Azi N, Gendreau M, Potvin J Y (2006). An exact algorithm for a single-vehicle routing problem with time windows and multiple routes. *European Journal of Operational Research* **178**: 755–766.

-
- Brandão J, Mercer A (1997). A tabu search algorithm for the multi-trip vehicle routing and scheduling problem. *European Journal of Operational Research* **100**: 180–191.
- Brandão J, Mercer A (1998). The multi-trip vehicle routing problem. *Journal of the Operational Research Society* **49**: 799–805.
- Cordeau J-F, Gendreau M, Laporte G (1997). A Tabu Search Heuristic for Periodic and Multi-Depot Vehicle Routing Problems. *Networks* **30**: 105–119.
- Cordeau J-F, Laporte G, Mercier A (2001). A unified tabu search heuristic for vehicle routing problems with time windows. *Journal of the Operational Research Society* **52**: 928–936.
- Cordeau J-F, Laporte G, Mercier A (2004). Improved tabu search algorithm for the handling of route duration constraints in vehicle routing problems with time windows. *Journal of the Operational Research Society* **55**: 542–546.
- Delgado C, Laguna M, Pacheco J (2005). Minimizing Labor Requirements in a Periodic Vehicle Loading Problem. *Computational Optimization and Applications* **32**: 299–320.
- Fagerholt K (1999). Optimal fleet design in a ship routing problem. *International Transactions in Operations Research* **6**: 453–464.
- Fagerholt K, Lindstad H (2000). Optimal policies for maintaining a supply service in the Norwegian Sea. *Omega* **28**: 269–275.
- Fleischmann B (1990). The vehicle routing problem with multiple use of vehicles. Working paper, Fachbereich Wirtschaftswissenschaften, Universität Hamburg.
- Gribkovskaia I, Laporte G, Shlopak A (2007). A tabu search heuristic for a routing problem arising in servicing of offshore oil and gas platforms. *Journal of the Operational Research Society* **59**: 1449–1459.
- Halvorsen-Weare E E, Fagerholt K, Nonås L M, Asbjørnslett B E (2009). Fleet size and mix and periodic routing of offshore supply vessels. Working paper, The Norwegian University of Science and Technology.
- Hemmelmayr V C, Doerner K F, Hartl R F (2009). A variable neighbourhood search heuristic for periodic routing problems. *European Journal of Operational Research* **195**: 791–802.
- Olivera A, Viera O (2007). Adaptive memory programming for the vehicle routing problem with multiple trips. *Computers & Operations Research* **34**: 28–47.
- Polacek M, Doerner K F, Hartl R F, Kiechle G and Reimann M (2007). Scheduling periodic customer visits for a traveling salesperson. *European Journal of Operational Research* **179**: 823–837.

Shaw P (1998). Using constraint programming and local search methods to solve vehicle routing problems. In: Principles and Practice of Constraint Programming - CP98, Lecture Notes in Computer Science (vol. 1520) 1998. Springer: Berlin / Heidelberg, pp 417–431.

Taillard É, Laporte G and Gendreau M (1996). Vehicle routing with multiple use of vehicles. *Journal of the Operational Research Society* **47**: 1065–1070.